# Unit 10: Functions

CONTENTS



## Objectives

After studying this unit, you will be able to:

- State the need for user defined functions
- Identify category of functions
- Describe functions that return multiple values
- Discuss recursive functions

## Introduction

A function is a programming unit with a unique name that may be identified. It can be invoked by a programme once it has been defined. When called, it may accept zero or more inputs. The code placed inside the function specification determines what should be done with the incoming input(s). The function generates a single output after doing the given transformation. The caller of the function receives this output.

## 10.1 Need for User-defined Function

Why write separate functions at all? Why not squeeze the entire logic into one function, main( )?

**Two reasons:**

1. Writing functions avoids rewriting the same code over and over. Suppose you have a section of code in your program that calculates area of a triangle. If later in the program you want to calculate the area of a different triangle, you won't like it if you are required to write the same instructions all over again. Instead, you would prefer to jump to a 'section of code' that calculates area and then jump back to the place from where you left off. This section of code is nothing but a function.

2. Using functions it becomes easier to write programs and keep track of what they are doing. If the operation of a program can be divided into separate activities, and each activity placed in a different function, then each could be written and checked more or less independently. Separating the code into modular functions also makes the program easier to design and understand.

## 10.2 A Multifunction Program

The use of a function is one of the advantages of the C programming language. Always behave like a standard function or method in C. One function may call another, and so on. In C, there are no limitations on the number of functions that can be called in a programme. It is preferable to decompose the complex problem into tiny, easily manageable parts and develop a function. The control will be passed from the calling programme part to the called function block. If the called function is successfully executed, control will be returned to the programme segment that called it. There is always overhead of a transfer of the control between calling portion and a called function block.

 **Example:**

Example: The multifunction program segment is shown below

```
function1 ()
{
        ----------
        ----------
function2 ();
        ----------
        ----------
function4 ();
}
function2 ()
{
        ----------
        ----------
function3 ();
        ----------
        ----------
}
function3 ();
{
        ----------
        ----------
}
function4 ()
{
        ----------
        ----------
}
```

**Lab Exercise:**

A program to demonstrate the transfer of control between the multifunction

program.

```
Main()
{
int j = 10;
printf (“Inside the main() function\n”);
function1 ();
printf (“after the function 1\n”);
printf (“main function () \n”);
printf (“j = %d\n”, j);
}
function1 ()
{
int i,n;
n = 3;
for(i = 0; i<=n-1; ++){
printf(“inside a function 1\n”);
printf(“i = %d\n”, i);
function2 ();
}
}
function2 ()
{
printf (“transfer of control\n);
printf (“inside a function 2\n”);
}
```

## 10.3 Elements of User-defined Functions

Functions are classified as one of the derived data types in C. We can therefore define functions and use them like any other variables in C programs. It is therefore not a surprise to note that there exist some similarities between functions and variables in C.

1. Both function names and variable names are considered identifiers and therefore they must adhere to the rules for identifiers.

2. Like variables, functions have types associated with them. 3. Like variables, function names and their must be declared and defined before they are used in a program.

In order to make use of a user-defined function, we need to establish three elements that are related to functions.

1. Function definition.

2. Function call

3. Function declaration.

The function definition is an independent program, module that is specially written to implement the requirements if the function. In order to use this function we need to invoke it is a required place in the program. This is known as the function call. The program that calls the function is referred to as calling program or calling function.

### Definition of Functions

A function is a standalone piece of executable code that can be called from any other function. The idea of functions comes to mind in many systems when a group of statements must be executed repeatedly at various points in the programme and possibly with different sets of data. Those repeating statements are stored in a function and called as needed. When a function is called, control is sent to the called function, which is then run, before being returned to the calling function (to the statement following the function call). Let us see an example as shown below:

 **Example:**

```
/* Program to illustrate a function*/
#include <stdio.h>
main ()
{
void sample( );
printf("\n You are in main");
}
void sample( )
{
printf("\n You are in sample");
}
```

Output:

```
You are in sample
You are in main
```

Here we are calling a function sample ( ) through main( ) i.e. control of execution transfers from main( ) to sample( ) , which means main( ) is suspended for some time and sample( ) is executed. After its execution the control returns back to main( ), at the statement following function call and the execution of main( ) is resumed.

The syntax of a function is:

```
return data type function_name (list of arguments)
{
datatype declaration of the arguments;
executable statements;
return (expression);
}
```

where,

1. Return data type is the same as the data type of the variable that is returned by the function using return statement.

2. A function_name is formed in the same way as variable names/identifiers are formed.

3. The list of arguments or parameters are valid variable names as shown below, separated by c

4. Arguments give the values which are passed from the calling function.

5. The body of function contains executable statements.

6. The return statement returns a single value to the calling function.ommas: (data type1 var1,data type2 var2,…….. data type n var n) for example (int x, float y, char z).

**Example**: Let us write a simple function that calculates the square of an integer.

```
/*Program to calculate the square of a given integer*/
/* square( ) function */
{
int square (int no) /*passing of argument */
int result ; /* local variable to function square */
result = no*no;
return (result); /* returns an integer value */
}
/*It will be called from main()as follows */
main( )
{
int n ,sq; /* local variable to function main */
printf (“Enter a number to calculate square value”);
scanf(“%d”,&n);
sq=square(n); /* function call with parameter passing */
printf (”\nSquare of the number is : %d”, sq);
} /* program ends */
```

Output:

Enter a number to calculate square value: 5

Square of the number is: 25

## 10.4 Return Value and their Types

If a function has to return a value to the calling function, it is done through the return statement. It may be possible that a function does not return any value; only the control is transferred to the calling function. The syntax for the return statement is:

return (expression);

We have seen in the square( ) function, the return statement, which returns an integer value.

### Important Points

1. You can pass any number of arguments to a function but can return only one value at a time.

**Example:** The following are the valid return statements

(a) return (5);

(b) return (x*y);

**Example**: The following are the invalid return statements

(a) return (2, 3);

(b) return (x, y);

2. If a function does not return anything, void specifier is used in the function declaration.

Example:

```
void square (int no)
{
int sq;
sq = no*no;
printf ("square is %d", sq);
}
```

3. All the function's return type is by default is "int", i.e. a function returns an integer value, if no type specifier is used in the function declaration.

Examples:

(a) square (int no); /* will return an integer value */

(b) int square (int no); /* will return an integer value */

(c) void square (int no); /* will not return anything */

4. What happens if a function has to return some value other than integer? The answer is very simple: use the particular type specifier in the function declaration.

Example: Consider the code fragments of function definitions below:

(a) Code Fragment – 1:

```
char func_char( …….. )
{
char c;
……
……
……
}
```

(b) Code Fragment – 1:

```
float func_float (……..)
{ fl
oat f;
……...
……...
……...
return(f);
}
```

Thus from the above examples, we see that you can return all the data types from a function, the only condition being that the value returned using return statement and the type specifier used in function declaration should match.

5. A function can have many return statements. This thing happens when some condition based returns are required.

 **Example:**

```
/*Function to find greater of two numbers*/
int greater (int x, int y)
{
if (x>y)
return (x);
else
return (y);
}
```

6. And finally, with the execution of return statement, the control is transferred to the calling function with the value associated with it.

In the above example, if we take x = 5 and y = 3, then the control will be transferred to the calling function when the first return statement will be encountered, as the condition (x > y) will be satisfied. All the remaining executable statements in the function will not be executed after this returning.

### Function Calls

A function can be called by supplying its name followed by a list of arguments separated by commas and surrounded in parentheses. If a function call does not require any parameters, it must be followed by an empty pair of parenthesis.

The arguments appearing in the function call are referred to as actual arguments, in contrast to the formal arguments that appear in the first line of function definition.

 **Lab Exercise:**

e.g.: /* Program to find square of given number */

```
main( )
{
float square (float); /* function prototype dec/n*/
float a, b;
printf (“\n Enter the number:”);
scanf (“%f”, &a);
b = square (a); /* calling of function with */
/ * actual arguments */
printf (“Square of entered no. is = %f” , b);
}
float square (x) / * function definition with format l argument * /
float x; /* format l argument declaration * /
{
float y; /* Local variable declaration
```

```
y = x * x;
return (y);
}
```

Output:

```
Enter the number: 2
Square of the entered number is = 4
```

### Function Declaration

A function is declared in the following manner:

```
<return_data_type> <function_name>(arg1, arg2, arg3)
<data_type_1> arg1; <data_type_2> arg2; <data_type_3> arg3;
{
statement-1;
statement-2
:
:
statement-n;
return(<expression of return_data_type>);
}
```

 **Example:** The following function (name being getsq) returns the square of the input number of float type. Clearly the <return_data_type> will also be float type.

```
float getsq(x)
float x;
{
return(x*x);
}
```

Another form of a function definition is:

```
<return_data_type><function_name>(formal argument list)
{
statement-1;
statement-2;
----------
statement-n;
return (<expression of return_data_type>);
}
```

Where formal argument list is a comma separated list of variables and their corresponding data types.

The following function, addthem, takes two int type arguments and returns the sum of the two.

```
int addthem(int a, int b)
{
```

return(a+b);

}

The <return_data_type> always represents the data type of the value which is returned. The type specification can be omitted if the function returns an integer or a character.

An empty pair of parenthesis must follow the function name if the function definition does not include any arguments.

The argument declarations follow the first line. Each formal argument must have the same data type as its corresponding actual argument.

The remainder of the function definition is a compound statement that defines the action to be taken by the function. It is referred to as the body of the function.

The last statement in the body of function is return (expression). It is used to return the computed result, if any, to the calling program.

## 10.5 Category of Functions

We categorize a function's invoking (calling) depending on arguments or parameters and their returning a value. In simple words, we can divide a function's invoking into four types

depending on whether parameters are passed to a function or not and whether a function returns some value or not.

The various types of calling functions are:

1. With no arguments and with no return value.

2. With no arguments and with return value.

3. With arguments and with no return value.

4. With arguments and with return value.

11.9 No Argument and no Return Values

Any function which has no arguments and does not return any values to the calling function, falls in this category. These type of functions are confined to themselves i.e. neither do they receive any data from the calling function nor do they transfer any data to the calling function.

So there is no data communication between the calling and the called function are only program control will be transferred.

 **Example:**

```
/* Program for illustration of the function with no arguments and no return
value*/
/* Function with no arguments and no return value*/
#include <stdio.h>
main()
{
void message();
printf("Control is in main\n");
message(); /* Type 1 Function */
printf("Control is again in main\n");
}
void message()
{
```

```
printf("Control is in message function\n");
} /* does not return anything */
```

Output:

Control is in main

Control is in message function

Control is again in main

### Argument but no Return Values

If a function includes arguments but does not return anything, it falls in this category. One way communication takes place between the calling and the called function.

Before proceeding further, first we discuss the type of arguments or parameters here. There are two types of arguments:

1. Actual arguments

2. Formal arguments

Let us take an example to make this concept clear:

 **Example:** Write a program to calculate sum of any three given numbers.

```
#include <stdio.h>
main()
{
int a1, a2, a3;
void sum(int, int, int);
printf("Enter three numbers: ");
scanf ("%d%d%d",&a1,&a2,&a3);
sum (a1,a2,a3); /* Type 3 function */
}
/* function to calculate sum of three numbers */
void sum (int f1, int f2, int f3)
{
int s;
s = f1+ f2+ f3;
printf ("\nThe sum of the three numbers is %d\n",s);
}
```

Output

Enter three numbers: 23 34 45

The sum of the three numbers is 102

Here f1, f2, f3 are formal arguments and a1, a2, a3 are actual arguments.

Thus we see in the function declaration, the arguments are formal arguments, but when values are passed to the function during function call, they are actual arguments.

### Arguments with Return Values

In this category, two-way communication takes place between the calling and called function i.e. a function returns a value and also arguments are passed to it. We modify above example according to this category.

**Example:**

Write a program to calculate sum of three numbers.

```
/*Program to calculate the sum of three numbers*/
#include <stdio.h>
main ( )
{
int a1, a2, a3, result;
int sum(int, int, int);
printf("Please enter any 3 numbers:\n");
scanf ("%d %d %d", & a1, &a2, &a3);
result = sum (a1,a2,a3); /* function call */
printf ("Sum of the given numbers is : %d\n", result);
}
/* Function to calculate the sum of three numbers */
int sum (int f1, int f2, int f3)
{
return(f1+ f2 + f3); /* function returns a value */
}
```

Output

```
Please enter any 3 numbers:
3 4 5
Sum of the given numbers is: 12
```

## 10.6 Functions that Return Multiple Values

The compiler will raise a compilation error if a function is not called with the required number of properly typed arguments. When calling a function, there are two methods for passing arguments to it: call by value and call by reference.

### Call by Value

Call by value means sending the values of the arguments to functions. When a single value is passed to a function via an actual argument, the value of the actual argument is copied into the function. Therefore, the value of the corresponding formal argument can be altered within the function, but the value of the actual argument within the calling routine will not change. This procedure for passing the value of an argument to a function is known as passing by value or call by value.

**Lab Exercise**

e.g.: /* A simple C program containing a function that alters the

value of its argument. */

```
#include <stdio.h>
main()
```

```
{
int a = 2;
printf("\na = %d (from main, before calling the function)",a);
modify(a);
printf("\na = %d (from main, after calling the function)",a);
}
modify (int a)
{
a * = 3;
printf("\na = %d (from the function, after being modified)",a);
return;
}
```

output: a = 2 (from main, before calling the function)

a = 6 (from the function, after being modified)

a = 2 (from main, after calling the function)

The original value of a (i.e.=2) is displayed when main is executed. This value is then passed to the function modify, where it is multiplied by three and the new value of the formal argument that is displayed within the function. Finally, the value of a within main (i.e., the actual argument) is again displayed, after control is transferred back to function main from function modify.

These results show that a is not altered within main, even though the corresponding value of a is changed within modify.

Passing an argument by value has certain advantages and disadvantages.

On the positive side, it allows a single valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed as a single variable, it protects the value of this variable from alterations within the function.

On the negative side, it prevents information from being transferred back to the calling portion of the program via arguments. Thus, passing by value is restricted to a one-way transfer of

information.

### Call by Reference

Call by reference means sending the addresses of the arguments to the called function. In this method the addresses of actual arguments in the calling function are copied into formal arguments of the called functions. Thus using these addresses we would have an access to the actual arguments and hence we would be able to manipulate them. Using a call by reference intelligently, it is possible to make a function return more than one value at a time, which involves the study of pointer.

### Function Prototype

Before defining the function, it is desired to declare the function along with its prototype. In function prototype, the return value of function, type, and number of arguments are specified.

The declaration of all functions statement should be first statement in

main( ).

The general form of function declaration using ANSI Prototype is

data_type function_name (type1 arg1, type2 arg2 - - - - );

where arg1, arg2. . . are the list of arguments.

Function prototypes are desirable because they facilitate error checking between calls to a function and corresponding function definition. They also help the compiler to perform automatic type

conversions on function parameters. When a function is called, actual arguments are automatically converted to the types in function definition using normal rules of assignment.

### Recursive Functions

Recursion is a process by which a function calls itself repeatedly, until some specified condition has been satisfied. The process is used for repetitive computation in which each action is stated in terms of previous result.

In order to solve a problem recursively, two conditions must be satisfied:

1. The problem must be written in recursive form.

2. The problem statement must include a stopping condition.

 **Example:** /*To calculate the factorial of an integer recursively * /

```
# include <stdio.h>
main( )
{
int n;
long int fact (int);
printf (“\n n = “);
scanf (“%d”, &n);
printf (“\n n! = % ld” fact (n));
}
long int fact (int n)
{
if (n < = 1)
return 1;
else
return (n * factorial (n-1));
}
```

### Library Functions

The C programming language comes with a set of standard library functions that perform a variety of useful tasks. Library functions implement all input and output operations (e.g., writing to the terminal) as well as all math operations (e.g., sine and cosine evaluation).

It is important to call the proper header file at the start of the programme in order to use a library function. All of the functions in the library in question have a header file that tells the programme their name, type, and number and type of arguments. The preprocessor statement calls a header file.#include <filename>

where filename represents the name of the header file.

A library function is accessed by simply writing the function name, followed by a list of arguments, which represent the information being passed to the function. The arguments must be enclosed in parentheses, and separated by commas: they can be constants, variables, or more complex expressions. Note that the parentheses must be present even when there are no arguments.

Library Functions in Different Header Files

C Header Files

<assert.h> Program assertion functions

| | |
|---|---|
| <ctype.h> | Character type functions |
| <locale.h> | Localization functions |
| <math.h> | Mathematics functions |
| <setjmp.h> | Jump functions |
| <signal.h> | Signal handling functions |
| <stdarg.h> | Variable arguments handling functions |
| <stdio.h> | Standard Input/Output functions |
| <stdlib.h> | Standard Utility functions |
| <string.h> | String handling functions |
| <time.h> | Date time functions |

## Summary

- In this unit, we learnt about "Functions": definition, declaration, prototypes, types, function calls datatypes and storage classes, types function invoking and lastly Recursion.
- All these subtopics must have given you a clear idea of how to create and call functions from other functions, how to send values through arguments, and how to return values to the called function.
- We have seen that the functions, which do not return any value, must be declared as "void", return type.
- A function can return only one value at a time, although it can have many return statements.
- A function can return any of the data type specified in 'C'.

## Keywords

***Call by Reference***: It means sending the addresses of the arguments to the called function.

***Data types***: It refers to the type of information while storage class refers to the life-time of a variable and its scope within the program.

***Function Call***: A function can be called by specifying its name followed by a list of arguments enclosed in parentheses and separated by commas.

***Return Statement***: Information is returned from the function to the calling portion of the program via return statement.

## Self Assessment

1. Which one is not a library function

A. printf ( )
B. scanf ( )
C. gets ( )
D. abc ( )

2. Function call is _____

A. Print the statement
B. Use header file in program
C. calling a function whenever it is required in a program

D. None of these

3. Functions are used to __

A. Enhances the logical clarity of the program.
B. Helps to avoid repeated programming across programs.
C. Helps to avoid repeating a set of statements many times.
D. All of above

4. Scope of variable in C is_____

A. local
B. global
C. intermediate
D. both a and b

5. Variable used inside function is called _____

A. Global variable
B. Local variable
C. Both a and b
D. None of these

6. In program a and b is a_____

```
int sum()
{
int a=15, b =20;
return x+y;
}
```

A. Global variable
B. Local variable
C. Intermediate variable
D. None of above

7. int x and int y is a_____

```
add(int x,int y)
{
   int z;
   z=x+y;
   printf("result is= %d",z);
}
```

A. formal Parameter
B. actual Parameter
C. intermediate
D. both a and b

8. Actual parameter is_____

A. parameters that appear in function calls.
B. parameters that appear in function definition.
C. local to the function definition
D. above all

9. Call by value and call by reference is part of _____

A. pointers
B. array
C. functions
D. loops

10. In program we can modify original value in

A. Call by value
B. Call by reference
C. above all
D. None of above

11. A function is called indirect recursive _____

A. if it calls the same function.
B. if it calls the another function.
C. Execute other function
D. Above all

12. Function which call itself is called_____

A. Static function
B. Auto function
C. Recursive function
D. above all

## Answers for Self Assessment

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1. | D | 2. | C | 3. | D | 4. | D | 5. | B |
| 6. | C | 7. | A | 8. | A | 9. | C | 10. | B |
| 11. | B | 12. | C | | | | | | |

## Review Questions

1. Takes two integer inputs and produces the remainder when the larger is divided by the smaller.

2. Swaps the two given integers.
3. What do you mean by function call.
4. Describe return value and their types.
5. Evaluates the following series for a specified n: 12 + 22 + 32 + 42 + ……..n2
6. A positive integer is entered through the keyboard. Write a function to obtain the prime factors of this number.
7. Write a function which receives a float and an int from main( ), finds the product of these two and returns the product which is printed through main( ).
8. Write a function that receives marks received by a student in 3 subjects and returns the average and percentage of these marks. Call this function from main( ) and print the results in main( ).
9. Given three variables x, y, z write a function to circularly shift their values to right. In other words if x = 5, y = 8, z = 10 after circular shift y = 5, z = 8, x =10 after circular shift y = 5, z = 8 and x = 10. Call the function with variables a, b, c to circularly shift values.
10. Write a function to compute the distance between two points and use it to develop another function that will compute the area of the triangle whose vertices are A(x1, y1), B(x2, y2), and C(x3, y3). Use these functions to develop a function which returns a value 1 if the point (x, y) lines inside the triangle ABC, otherwise a value 0.
11. Write a function to find the binary equivalent of a given decimal integer and display it.

## Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C

## Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

www.programiz.com